Docker Basics – Hands-on Learning Documentation

Topic: Docker Fundamentals and Hands-on Practice
Purpose: To understand Docker commands, images, containers, and Docker Hub workflow.

---

1. What is Docker?

Docker is a platform that allows us to package and run applications inside containers.

A container contains the application and the dependencies required to run it.

Why do we use Docker?

Imagine that an application works on one developer's computer but fails on another computer because of different software versions or dependencies.

Docker helps solve this problem by providing a consistent environment for running applications.

Important Docker Concepts

Concept	Meaning
Docker Image	A template used to create containers.
Docker Container	A running or stopped instance of an image.
Dockerfile	A file containing instructions to build an image.
Docker Hub	An online registry for storing and sharing Docker images.
Docker Engine	The software responsible for building and running containers.
Docker CLI	The command-line interface used to communicate with Docker.

Simple example:

- Dockerfile → Instructions
- Image → Prepared template
- Container → Running instance created from the template
- Docker Hub → Online storage for images

Important: A container is not a complete virtual machine. It is an isolated environment that shares the host operating system's kernel. A container may look like a mini operating system, but internally it works differently from a virtual machine.

---

2. Check Docker Installation

2.1 Check Docker Version

```
docker --version
```

Purpose: Displays the installed Docker CLI version.

Example output:

Docker version 28.x.x, build ...

The actual version depends on your installation.

2.2 Check Docker Information

```
docker info
```

Purpose: Displays detailed information about Docker Engine.

It can show:

- Number of containers
- Number of images
- Running containers
- Docker storage driver
- Operating system information
- CPU and memory information
- Docker Engine configuration

Remember:

- docker --version checks the CLI version.
- docker info displays information about the Docker Engine.

If docker info fails with a daemon connection error, check whether Docker Desktop or the Docker Engine is running.

---

3. Understand Docker Images

A Docker image is a template used to create containers.

Images can be downloaded from Docker Hub or built locally using a Dockerfile.

3.1 Download an Image

```
docker pull alpine:latest
```

Purpose: Downloads the Alpine Linux image from the configured Docker registry, normally Docker Hub.

Here:

- docker pull → Downloads an image.
- alpine → Image name.
- latest → Image tag.

A tag identifies a particular image version or variant. The latest tag does not necessarily mean the newest software version.

3.2 List Downloaded Images

```
docker images
```

Purpose: Displays images available locally.

Example output:

```
REPOSITORY   TAG     IMAGE ID      CREATED      SIZE
alpine       latest  abc123...     ...          ...
node         latest  def456...     ...          ...
```

Important columns:

- REPOSITORY → Image name
- TAG → Image version or label
- IMAGE ID → Image identifier
- SIZE → Image size

Remember: docker images lists images, not containers.

---

4. Run a Docker Container

4.1 Run an Image

```
docker run alpine:latest
```

Purpose: Creates a container from the Alpine image and starts it.

However, Alpine normally does not run a long-lived foreground process by default. If no command is provided, the container may exit immediately.

Check the container using:

docker ps -a

4.2 Run an Interactive Container

```
docker run -it alpine:latest /bin/sh
```

Purpose: Starts an Alpine container and opens an interactive shell.

Here:

- docker run → Creates and starts a container.
- -i → Keeps standard input open.
- -t → Allocates a terminal.
- /bin/sh → Starts the shell inside the container.

Now you can execute commands inside the container:

ls
pwd
cd /tmp

To leave the shell:

exit

If the shell was the container's main process, exiting it will stop the container.

Important: Alpine commonly uses /bin/sh. It may not contain /bin/bash.

---

5. List Docker Containers

5.1 List Running Containers

```
docker ps
```

Purpose: Displays containers that are currently running.

5.2 List All Containers

```
docker ps -a
```

Purpose: Displays running and stopped containers.

Important columns:

- CONTAINER ID → Unique container identifier
- IMAGE → Image used to create the container
- COMMAND → Main command executed by the container
- STATUS → Current container status
- NAMES → Container name

Difference:

Command	Result
docker ps	Running containers
docker ps -a	All containers, including stopped ones
docker images	Locally available images

---

6. Create a Simple Docker Image Using a Dockerfile

A Dockerfile contains instructions used to build a Docker image.

Step 1: Create a Project Folder

For example:

docker-hello/

Open the folder and create a file named exactly:

Dockerfile

Make sure it is not accidentally saved as Dockerfile.txt.

Step 2: Write the Dockerfile

FROM alpine:latest

CMD ["echo", "Hello, My Docker Hands On!"]

Step 3: Understand Each Instruction

FROM alpine:latest

Specifies the base image for the new image.

CMD ["echo", "Hello, My Docker Hands On!"]

Specifies the default command that runs when a container starts without another command overriding it.

In this example, the container prints a message and exits.

Step 4: Build the Image

Run this command from inside the docker-hello folder:

```
docker build -t my-hello-image .
```

Understand the command:

- docker build → Builds an image.
- -t → Assigns a name and optionally a tag.
- my-hello-image → Name of the image.
- . → Use the current directory as the build context.

Important correction: The build command needs a build context, such as . at the end.

Step 5: Verify the Image

```
docker images
```

You should find my-hello-image in the output.

Step 6: Run the Image

```
docker run --name hello-container my-hello-image
```

Expected output:

Hello, My Docker Hands On!

The container prints the message and exits.

Check its status:

docker ps -a

You should find hello-container with an exited status.

Step 7: Run the Image Again

```
docker run --name hello-container-2 my-hello-image
```

The same image can create multiple containers.

Key learning: One image can be used to create many independent containers.

---

7. Container Name and Rename

7.1 Assign a Name When Creating a Container

```
docker run --name my-node-container node:latest
```

Purpose: Creates a container named my-node-container using the Node.js image.

This particular command runs Node.js without an interactive terminal. Depending on the image's default command and input state, it may exit. Use the following command if you want an interactive Node.js session.

docker run -it --name my-node-container-2 node:latest

You should see the Node.js prompt:

Welcome to Node.js ...
>

You can now type JavaScript expressions:

console.log("Hello Docker");

To exit Node.js:

.exit

7.2 Rename a Container

```
docker rename my-node-container node-container
```

Purpose: Changes the name of an existing container.

The syntax is:

docker rename <old-name> <new-name>

Important: The --name option sets the container name when creating a container. docker rename changes the name after the container exists.

A container name is not the same as an image name.

---

8. Start and Stop Containers

8.1 Start a Stopped Container

```
docker start my-node-container
```

Purpose: Starts an existing stopped container.

You can also use its container ID:

docker start <container-id>

Replace the placeholder with the actual container ID.

Important: docker start reuses an existing container. It does not create a new container.

8.2 Stop a Running Container

```
docker stop my-node-container
```

Purpose: Gracefully stops a running container.

You can also use:

docker stop <container-id>

Docker normally sends a termination signal and waits for the process to stop before forcing termination.

8.3 Verify the Status

docker ps
docker ps -a

Use docker ps to check running containers and docker ps -a to check stopped containers too.

Remember:

- docker run → Creates and starts a new container.
- docker start → Starts an existing container.
- docker stop → Stops a running container.

---

9. Execute Commands Inside a Running Container

The docker exec command allows you to execute a command inside an already running container.

9.1 Open a Shell

```
docker exec -it <container-name> /bin/bash
```

Replace <container-name> with the actual container name or ID.

For example:

docker exec -it my-node-container-2 /bin/bash

This works only if the container is running and Bash is installed.

For containers that have only sh, use:

docker exec -it <container-name> /bin/sh

9.2 Explore the Container

Inside the container, try:

ls
pwd
cd /usr/local/bin
ls

For a Node.js image, you may find programs such as:

node
npm
npx

The exact directory contents depend on the image.

9.3 Open the Node.js REPL

Inside the container:

node

Now enter:

console.log("Hello from Node.js inside Docker!");

To exit the Node.js REPL:

.exit

To leave the container shell:

exit

9.4 Important Difference Between run and exec

Command	Purpose
docker run	Creates and starts a new container
docker exec	Runs an additional command inside an existing running container

Important: docker exec requires the container to be running. If the container is stopped, start it first.

---

10. Copy Files Between the Host and a Container

Docker provides the docker cp command to copy files and directories between your computer and a container.

Here, the **host** means your own computer, and the **container** means the Docker environment.

10.1 Copy a File From Your Computer to a Container

Suppose you have a file named trail.txt in your current Windows directory.

```
docker cp .\trail.txt my-alpine-container:/example.txt
```

Purpose: Copies trail.txt from your Windows computer into the container as /example.txt.

Explanation:

- .\trail.txt → Source file on Windows.
- my-alpine-container → Existing container name.
- /example.txt → Destination path inside the container.

Verify the file inside the container:

docker exec -it my-alpine-container /bin/sh

Then:

ls /
cat /example.txt

10.2 Copy a File From a Container to Your Computer

```
docker cp my-alpine-container:/example.txt .\copy.txt
```

Purpose: Copies /example.txt from the container to a local file named copy.txt in the current Windows directory.

Important Notes

- docker cp can copy files to or from a running or stopped container.
- The destination directory must exist.
- The source file or directory must exist.
- Replace the example container name with the name of your actual container.

Remember: docker cp copies files; it does not move them.

---

11. Docker Login and Logout

Docker Hub is an online service where users can store and share Docker images.

11.1 Login to Docker Hub

```
docker login
```

Purpose: Authenticates your Docker CLI with a container registry, normally Docker Hub.

Depending on the setup, Docker may ask you to authenticate through a browser or enter credentials.

11.2 Logout

```
docker logout
```

Purpose: Removes the locally stored authentication for the default registry.

You generally log in before pushing private or public images to Docker Hub.

Security note: Never put passwords, access tokens, or other credentials directly into a Dockerfile or public source-code repository.

---

12. Tag a Docker Image for Docker Hub

Before pushing an image to Docker Hub, give it a repository name that includes your Docker Hub username.

Suppose your Docker Hub username is muthu0506 and your local image is named my-hub-image.

12.1 Tag the Image

```
docker tag my-hub-image muthu0506/my-hub-image:latest
```

Purpose: Creates another image tag pointing to the same local image.

Understand the format:

<docker-hub-username>/<repository-name>:<tag>

For this example:

- muthu0506 → Docker Hub username
- my-hub-image → Repository name
- latest → Image tag

Important: docker tag does not upload the image. It prepares the local image reference for pushing.

12.2 Verify the Tag

docker images

You should see the tagged image:

muthu0506/my-hub-image latest

The original local tag can still exist.

---

13. Push a Docker Image to Docker Hub

Step 1: Login

```
docker login
```

Step 2: Tag the Image

```
docker tag my-hub-image muthu0506/my-hub-image:latest
```

Step 3: Push the Image

```
docker push muthu0506/my-hub-image:latest
```

Purpose: Uploads the tagged image to Docker Hub.

If the push succeeds, Docker uploads any image layers that the registry does not already have.

Step 4: Verify on Docker Hub

Open Docker Hub in your browser and check your repositories.

You should find the repository and the uploaded tag if the push was successful.

Important: The repository must exist or your account must have permission to create and push to it, depending on the registry setup. You must also have permission to push to the target namespace.

Understanding latest and Custom Tags

You can use a custom version tag:

docker tag my-hub-image muthu0506/my-hub-image:v1.0
docker push muthu0506/my-hub-image:v1.0

A custom tag makes it easier to identify a specific release.

---

14. Pull an Image From Docker Hub

Suppose an image has already been uploaded to Docker Hub.

Step 1: Pull the Image

```
docker pull muthu0506/my-hub-image:latest
```

Purpose: Downloads the specified image tag to your local machine.

Step 2: Verify the Download

```
docker images
```

Step 3: Run a Container From the Downloaded Image

```
docker run --name downloaded-container muthu0506/my-hub-image:latest
```

If this is the earlier Alpine image with the echo command, it will print the configured message and exit.

Key learning: You can pull an image from Docker Hub and run it without rebuilding it from the original Dockerfile.

---

15. Complete Docker Workflow

The following is the typical workflow for a simple Docker image.

Build and Run Locally

1. Write a Dockerfile.
2. Build the image.
3. Verify the image.
4. Run a container from the image.
5. Inspect the container's output and status.

Upload to Docker Hub

1. Log in to Docker Hub.
2. Tag the image with your Docker Hub username and repository.

3. Push the tagged image.
4. Verify that the image appears in Docker Hub.

Download and Run on Another Machine

1. Log in if authentication is required.
2. Pull the image from Docker Hub.
3. Run a container from the downloaded image.

Workflow Diagram

Dockerfile
  |
  v
docker build
  |
  v
Local Docker Image
  |
  v
docker run
  |
  v
Docker Container

Local Docker Image
  |
  v
docker tag
  |
  v
docker push
  |
  v
Docker Hub
  |
  v
docker pull
  |
  v
Local Docker Image
  |
  v
docker run
  |
  v
New Docker Container

Remember: Docker Hub stores images, not the live running state of your container. Pushing an image does not upload your container's current running process or all its writable-layer changes.

---

16. Quick Command Reference

Command	Purpose
docker --version	Check Docker CLI version
docker info	Display Docker Engine information
docker pull IMAGE	Download an image
docker images	List local images
docker build -t NAME .	Build an image
docker run IMAGE	Create and start a container
docker run -it IMAGE /bin/sh	Open an interactive shell
docker ps	List running containers
docker ps -a	List all containers
docker start CONTAINER	Start an existing container
docker stop CONTAINER	Stop a running container
docker rename OLD NEW	Rename a container
docker exec -it CONTAINER /bin/sh	Open a shell in a running container
docker cp SOURCE DESTINATION	Copy files between host and container
docker login	Authenticate with a registry
docker logout	Log out of the default registry
docker tag SOURCE TARGET	Assign another image tag
docker push IMAGE:TAG	Upload an image
docker pull IMAGE:TAG	Download an image

Replace placeholders such as IMAGE, CONTAINER, SOURCE, and DESTINATION with actual names or paths. Do not type the angle-bracket placeholders literally.

---

17. Common Beginner Mistakes

Mistake 1: Using `docker build` without a build context

Incorrect:

```
docker build -t my-hello-image
```

Correct:

```
docker build -t my-hello-image .
```

Mistake 2: Trying to use Bash in an image that does not contain Bash

Use `/bin/sh` if Bash is unavailable.

Mistake 3: Using `docker exec` on a stopped container

Start the container first, provided its configured main process stays running.

Mistake 4: Confusing image names and container names

The image is the template; the container is the instance created from it. They can have different names.

Mistake 5: Assuming `docker run` always leaves a container running

A container stops when its main process finishes. A command such as `echo` prints a message and exits, so the container normally stops immediately afterward.

Mistake 6: Pushing an image without tagging it correctly

Use the Docker Hub username, repository name, and tag in the target image reference.

Mistake 7: Assuming `latest` always means the newest version

It is a tag name, not an automatic guarantee of version freshness.

---

18. Hands-on Practice Checklist

Use this checklist to verify that you understand the concepts.

- Check Docker version and Docker Engine information.
- Pull the Alpine image.
- List local images.

- Run an interactive Alpine container.
- Compare docker ps and docker ps -a.
- Create a Dockerfile.
- Build an image using docker build.
- Run a container from your custom image.
- Start and stop an existing container.
- Open a shell using docker exec.
- Copy a file into a container and back to Windows.
- Log in to Docker Hub.
- Tag a local image.
- Push the image to Docker Hub.
- Pull the image again and run it.

---

19. Final Summary

Docker helps developers build, package, share, and run applications in containers.

The most important concepts learned in this hands-on session are:

1. **Images** are templates for containers.
2. **Containers** are isolated environments created from images.
3. **Dockerfiles** contain instructions for building images.
4. **Docker commands** help manage images and containers.
5. **Docker Hub** allows images to be shared online.
6. **Tagging** gives an image a registry-compatible name and version.
7. **Push and pull** transfer images between a local machine and a registry.
8. **docker run and docker exec** have different purposes.
9. A container can stop even when its image remains available.
10. The same image can be used to create containers on different machines.

My learning takeaway: I now understand the basic Docker workflow, from creating an image using a Dockerfile to running containers and sharing images through Docker Hub.

Additional commands:

- taskkill /IM "Docker Desktop.exe" /F
- wsl --shutdown
- wsl --list --running

Resources:

- **Docker for Beginners in Tamil | Docker Hub | 1 Hour Full Tutorial - YouTube**
- **Docker from Scratch - Medium**